Mantenimiento de Gilded Rose

Actividad 2, Unidad 2

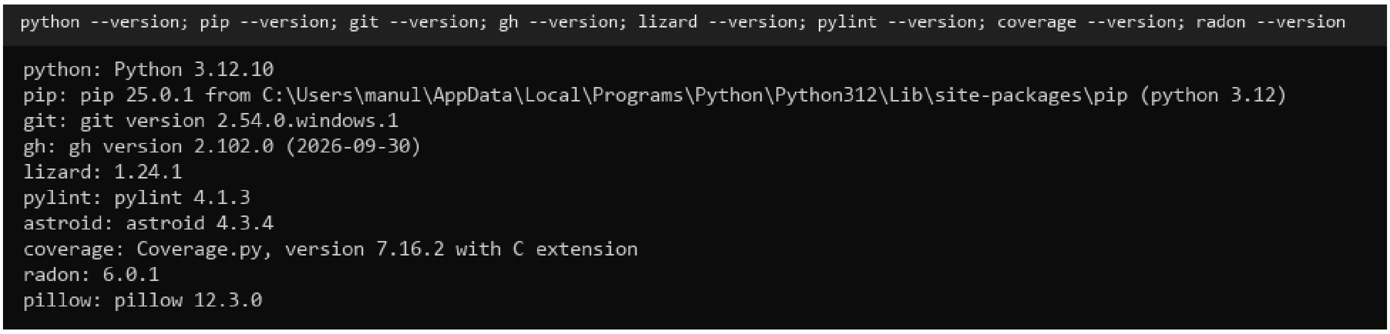
Manuel Lozano

17 de octubre de 2026

1. Introducción y selección del código

El caso de clase es ConstruYA, una cadena de ferreterías cuyo inventario acumula deuda técnica: un cambio, como integrar una pasarela de pagos, tarda más y puede introducir fallas. El componente intervenido es el inventario de la tienda Gilded Rose, kata de Emily Bache en Python (`https://github.com/emilybache/GildedRose-Refactoring-Kata`). Cumple el mismo papel: una regla de negocio nueva es una solicitud de cambio. La licencia original se conserva. La clase `Item` y la propiedad `items` no se modifican.

El código que no es prueba suma 80 líneas: 46 en `gilded_rose.py` y 34 en `texttest_fixture.py`. El conteo está en la evidencia. No llega al mínimo de 150 que pide el procedimiento. No se añadió código de relleno. La figura 1 muestra las versiones del entorno, renderizadas desde esa evidencia.



```
python --version; pip --version; git --version; gh --version; lizard --version; pylint --version; coverage --version; radon --version

python: Python 3.12.10
pip: pip 25.0.1 from C:\Users\manul\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip (python 3.12)
git: git version 2.54.0.windows.1
gh: gh version 2.102.0 (2026-09-30)
lizard: 1.24.1
pylint: pylint 4.1.3
astroid: astroid 4.3.4
coverage: Coverage.py, version 7.16.2 with C extension
radon: 6.0.1
pillow: pillow 12.3.0
```

Figura 1: Salida de los comandos de versión (Python, pip, git, gh, lizard, pylint, coverage, radon y pillow), tomada de `evidencia/entorno.txt`.

2. Diagnóstico

pylint 4.1.3, sobre `gilded_rose.py` y `texttest_fixture.py`, calificó el original con 6.48/10 (figura 2). No marca método largo, números mágicos ni clones. lizard 1.24.1 con `-C 10 -L 50` sí marca `update_quality`: NLOC 29, CCN 19, longitud 29 (figura 3). La longitud no supera 50; el aviso es por la complejidad.

Smell	Ubicación	Evidencia	Técnica
Método con demasiadas ramas	`update_quality`, 8–36	lizard CCN 19. pylint R0912: 18/12	Extraer método y estrategia
Condicionales anidados	líneas 9–36	pylint R1702: 6/5, tres veces	Guardas y retorno temprano
Números mágicos	0, 50, 11 y 6	pylint no los marca; inspección	Constantes con nombre
Cadenas repetidas	Brie, Sulfuras, Backstage	pylint R1714 solo en la línea 10	Constantes con nombre
Duplicación al subir o bajar calidad	líneas 13, 16, 20, 23, 31 y 36	sin detector de clones; inspección	Métodos con límite
Tipo de producto como cadena	comparaciones de `name`	ninguna herramienta lo nombra	Despacho por estrategia

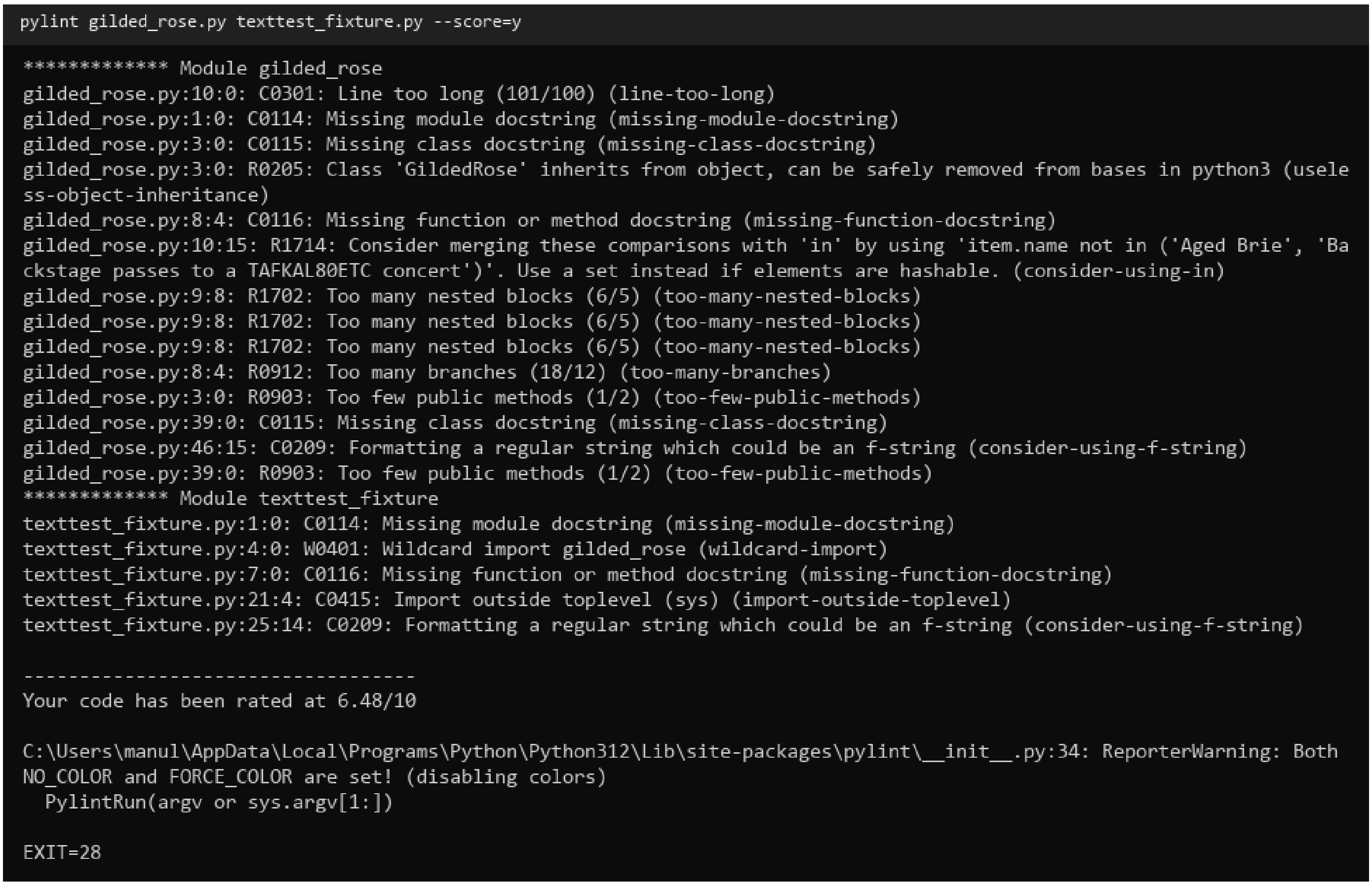
3. Refactorización

Cinco técnicas, un commit cada una, en `refactor/gilded-rose`. Tras cada una, las 13 pruebas siguieron en verde. El golden master no se regeneró.

Antes, la regla y el límite vivían en el mismo bloque:

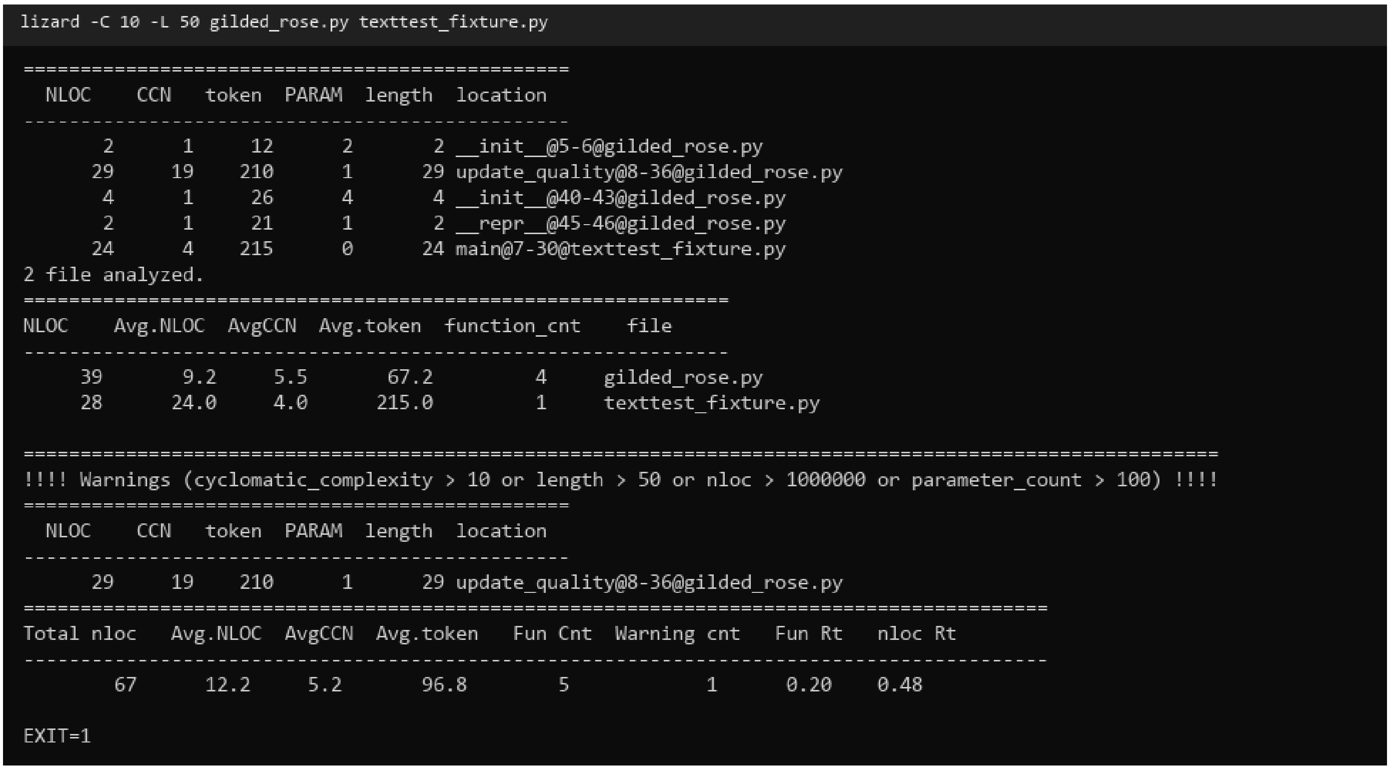
```
if item.quality > 0:
    if item.name != "Sulfuras, Hand of Ragnaros":
        item.quality = item.quality - 1
```

Después, el límite está en una función y el tipo se elige en un diccionario (`actualizadores.py`):



```
pylint gilded_rose.py texttest_fixture.py --score=y

************* Module gilded_rose
gilded_rose.py:10:0: C0301: Line too long (101/100) (line-too-long)
gilded_rose.py:1:0: C0114: Missing module docstring (missing-module-docstring)
gilded_rose.py:3:0: C0115: Missing class docstring (missing-class-docstring)
gilded_rose.py:3:0: R0205: Class 'GildedRose' inherits from object, can be safely removed from bases in python3 (usele
ss-object-inheritance)
gilded_rose.py:8:4: C0116: Missing function or method docstring (missing-function-docstring)
gilded_rose.py:10:15: R1714: Consider merging these comparisons with 'in' by using 'item.name not in ('Aged Brie', 'Ba
ckstage passes to a TAFKAL80ETC concert')'. Use a set instead if elements are hashable. (consider-using-in)
gilded_rose.py:9:8: R1702: Too many nested blocks (6/5) (too-many-nested-blocks)
gilded_rose.py:9:8: R1702: Too many nested blocks (6/5) (too-many-nested-blocks)
gilded_rose.py:9:8: R1702: Too many nested blocks (6/5) (too-many-nested-blocks)
gilded_rose.py:8:4: R0912: Too many branches (18/12) (too-many-branches)
gilded_rose.py:3:0: R0903: Too few public methods (1/2) (too-few-public-methods)
gilded_rose.py:39:0: C0115: Missing class docstring (missing-class-docstring)
gilded_rose.py:46:15: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
gilded_rose.py:39:0: R0903: Too few public methods (1/2) (too-few-public-methods)
************* Module texttest_fixture
texttest_fixture.py:1:0: C0114: Missing module docstring (missing-module-docstring)
texttest_fixture.py:4:0: W0401: Wildcard import gilded_rose (wildcard-import)
texttest_fixture.py:7:0: C0116: Missing function or method docstring (missing-function-docstring)
texttest_fixture.py:21:4: C0415: Import outside toplevel (sys) (import-outside-toplevel)
texttest_fixture.py:25:14: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)

-----------------------------------
Your code has been rated at 6.48/10

C:\Users\manul\AppData\Local\Programs\Python\Python312\Lib\site-packages\pylint\__init__.py:34: ReporterWarning: Both
NO_COLOR and FORCE_COLOR are set! (disabling colors)
  PylintRun(argv or sys.argv[1:])

EXIT=28
```

Figura 2: Salida de `pylint gilded_rose.py texttest_fixture.py -score=y` antes de refactorizar.



```
lizard -C 10 -L 50 gilded_rose.py texttest_fixture.py

================================================
  NLOC    CCN   token  PARAM  length  location
------------------------------------------------
       2      1     12      2       2 __init__@5-6@gilded_rose.py
      29     19    210      1      29 update_quality@8-36@gilded_rose.py
       4      1     26      4       4 __init__@40-43@gilded_rose.py
       2      1     21      1       2 __repr__@45-46@gilded_rose.py
      24      4    215      0      24 main@7-30@texttest_fixture.py
2 file analyzed.
==============================================================
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt    file
--------------------------------------------------------------
     39       9.2     5.5       67.2         4     gilded_rose.py
     28      24.0     4.0      215.0         1     texttest_fixture.py

===========================================================================================================
!!!! Warnings (cyclomatic_complexity > 10 or length > 50 or nloc > 1000000 or parameter_count > 100) !!!!
================================================
  NLOC    CCN   token  PARAM  length  location
------------------------------------------------
      29     19    210      1      29 update_quality@8-36@gilded_rose.py
==========================================================================================
Total nloc   Avg.NLOC  AvgCCN  Avg.token   Fun Cnt  Warning cnt   Fun Rt   nloc Rt
------------------------------------------------------------------------------------------
        67      12.2     5.2       96.8        5            1      0.20    0.48

EXIT=1
```

Figura 3: Salida de `lizard -C 10 -L 50 gilded_rose.py texttest_fixture.py`.

```
def bajar_calidad(item):
    if item.quality > MIN_QUALITY:
        item.quality = item.quality - 1

DESPACHO = {
    AGED_BRIE: actualizar_brie,
    BACKSTAGE_PASSES: actualizar_backstage,
    SULFURAS: actualizar_sulfuras,
}
```

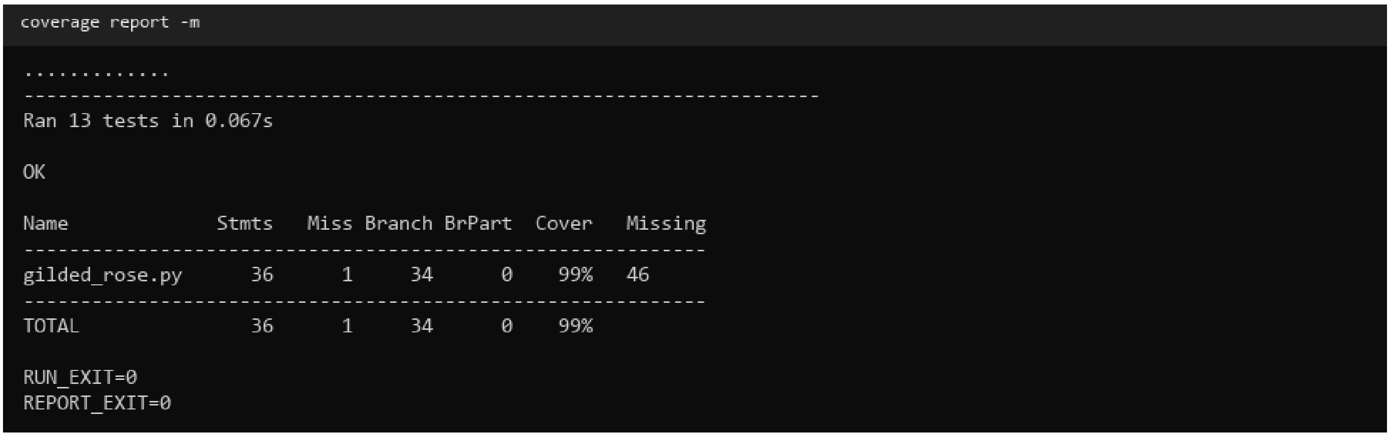
`update_quality` quedó en 3 líneas de código, CCN 2 y longitud 3 (figura 7). Las otras técnicas fueron constantes con nombre, métodos por tipo de ítem y cláusulas de guarda. Los fragmentos están en `evidencia/refactor_1_antes.txt` a `refactor_5_despues.txt`.

4. Herramientas de análisis estático y dinámico

La estática es pylint, complementada con lizard. La dinámica es coverage.py con `-branch`. No se usa la cobertura como prueba de que el diseño mejoró: con 5833 combinaciones y 40 días, el original ya cubría todas las ramas ejecutables.

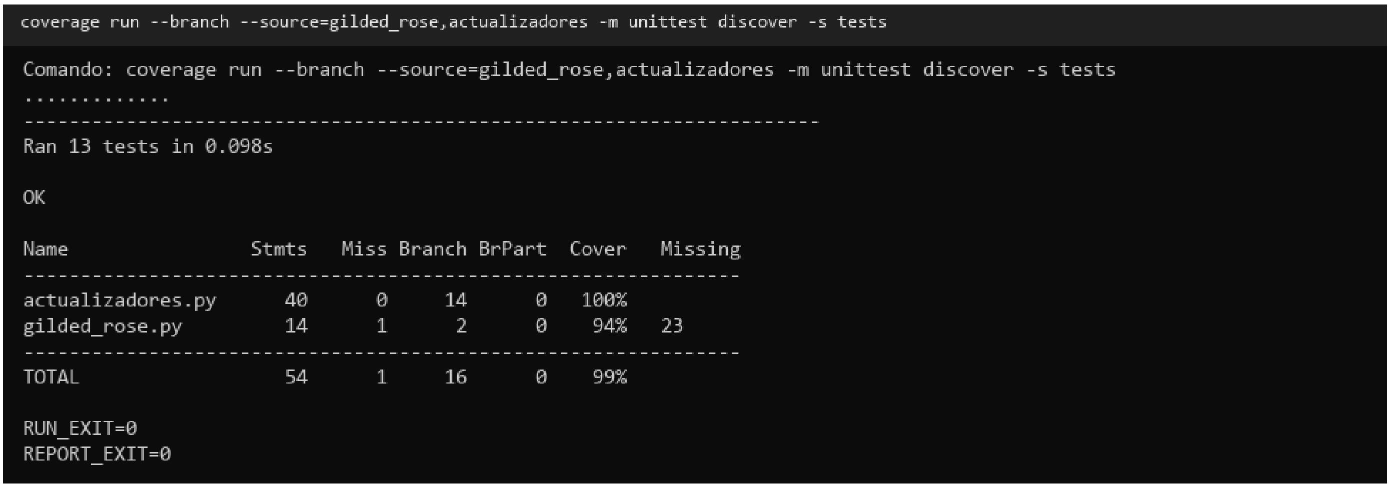
Métrica	Antes	Después
`update_quality`	NLOC 29, CCN 19, longitud 29	NLOC 3, CCN 2, longitud 3
CCN media (lizard)	5,2; una función sobre el umbral	1,9; ninguna sobre el umbral
pylint	6,48/10; 19 avisos	6,94/10; 22 avisos
Cobertura	36 sentencias, 1 sin cubrir, 34 ramas, 99 %	54 sentencias, 1 sin cubrir, 16 ramas, 99 %
Pruebas	13, todas OK	13, todas OK

Los avisos de pylint suben de 19 a 22 porque las funciones nuevas no tienen docstring y el import estrella marca `actualizar_item`. Desaparecen R1702 y R0912. Las ramas bajan de 34 a 16 porque se eliminó código duplicado, no porque las pruebas dejen de recorrer reglas. La línea sin cubrir sigue siendo `__repr__`. Figuras 4 y 5.



```
coverage report -m

.............
----------------------------------------------------------------------
Ran 13 tests in 0.067s

OK

Name             Stmts   Miss Branch BrPart  Cover   Missing
------------------------------------------------------------
gilded_rose.py      36      1     34      0    99%   46
------------------------------------------------------------
TOTAL               36      1     34      0    99%

RUN_EXIT=0
REPORT_EXIT=0
```

Figura 4: Salida de `coverage report -m` antes de refactorizar.

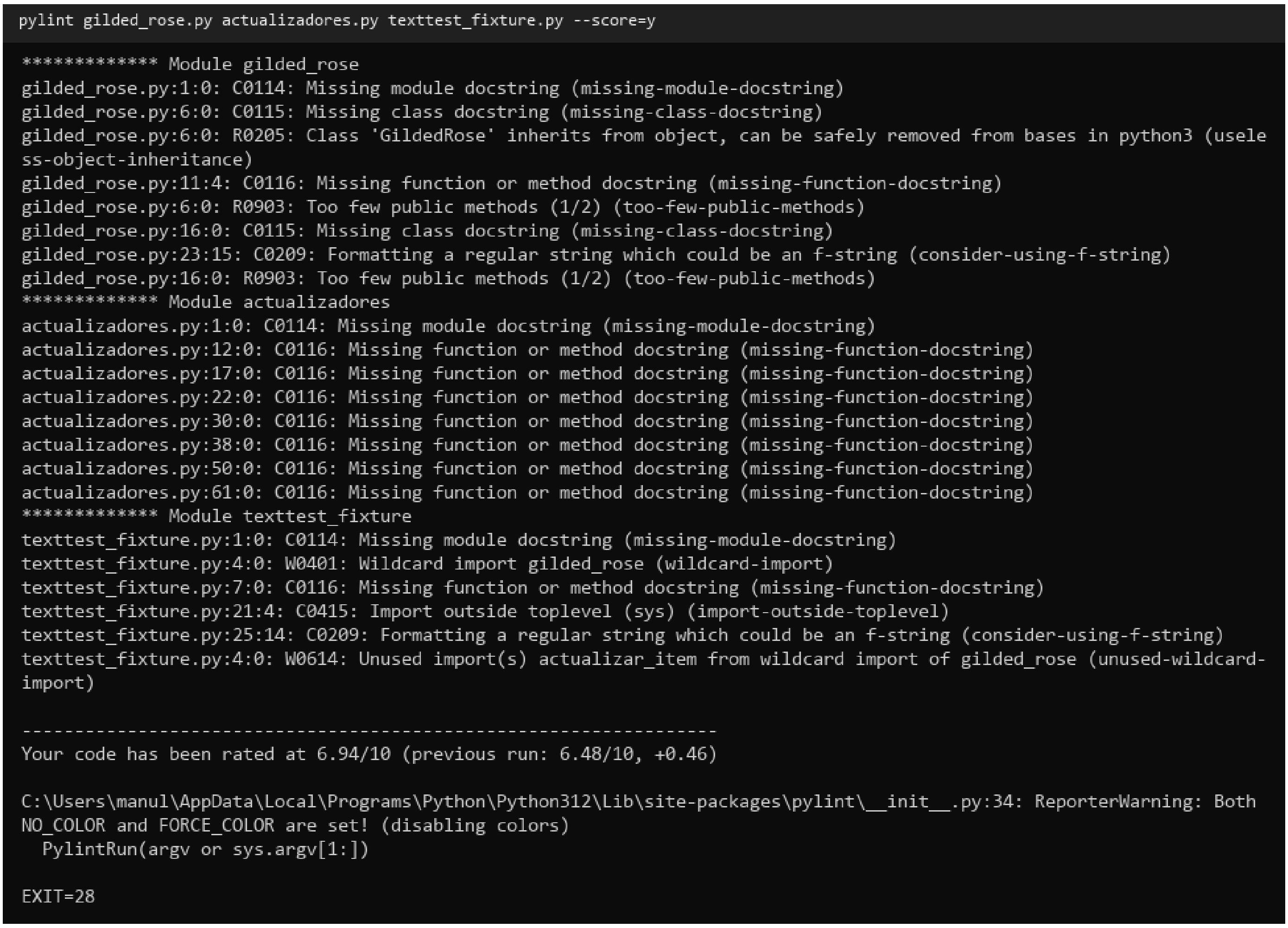


```
coverage run --branch --source=gilded_rose,actualizadores -m unittest discover -s tests

Comando: coverage run --branch --source=gilded_rose,actualizadores -m unittest discover -s tests
.............
----------------------------------------------------------------------
Ran 13 tests in 0.098s

OK

Name                Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------
actualizadores.py      40      0     14      0   100%
gilded_rose.py         14      1      2      0    94%   23
---------------------------------------------------------------
TOTAL                  54      1     16      0    99%

RUN_EXIT=0
REPORT_EXIT=0
```

Figura 5: Salida de `coverage run -branch -source=gilded_rose,actualizadores` y del reporte posterior.

5. Control de versiones

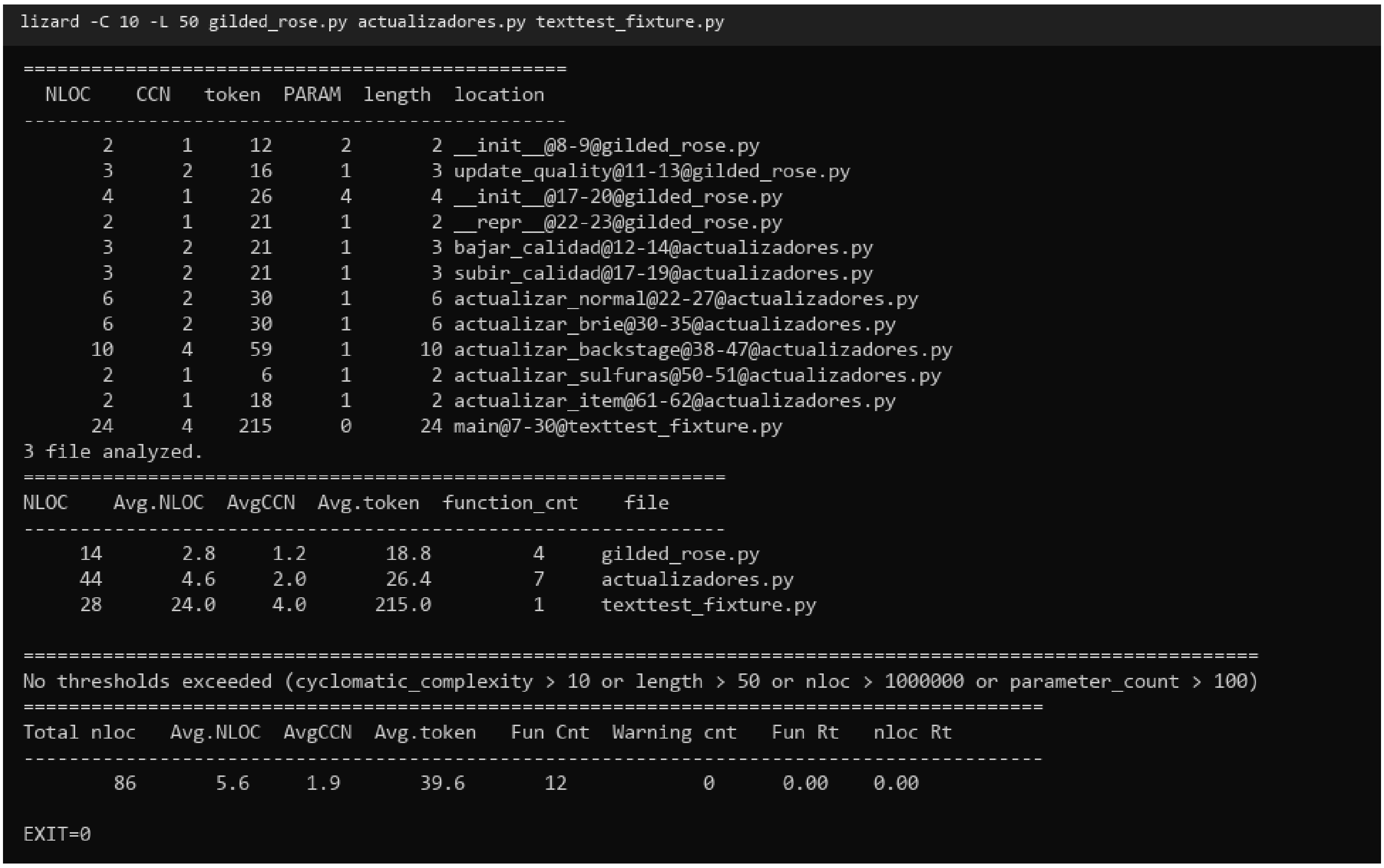
Repositorio público: `https://github.com/Manulol1904/gilded-rose-mantenimiento`. Rama `main`. La refactorización vivió en `refactor/gilded-rose`. La integración fue un merge `-no-ff`, commit `5dbd777`, mensaje `merge: integrar`



```
pylint gilded_rose.py actualizadores.py texttest_fixture.py --score=y
```

```
************* Module gilded_rose
gilded_rose.py:1:0: C0114: Missing module docstring (missing-module-docstring)
gilded_rose.py:6:0: C0115: Missing class docstring (missing-class-docstring)
gilded_rose.py:6:0: R0205: Class 'GildedRose' inherits from object, can be safely removed from bases in python3 (usele
ss-object-inheritance)
gilded_rose.py:11:4: C0116: Missing function or method docstring (missing-function-docstring)
gilded_rose.py:6:0: R0903: Too few public methods (1/2) (too-few-public-methods)
gilded_rose.py:16:0: C0115: Missing class docstring (missing-class-docstring)
gilded_rose.py:23:15: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
gilded_rose.py:16:0: R0903: Too few public methods (1/2) (too-few-public-methods)
************* Module actualizadores
actualizadores.py:1:0: C0114: Missing module docstring (missing-module-docstring)
actualizadores.py:12:0: C0116: Missing function or method docstring (missing-function-docstring)
actualizadores.py:17:0: C0116: Missing function or method docstring (missing-function-docstring)
actualizadores.py:22:0: C0116: Missing function or method docstring (missing-function-docstring)
actualizadores.py:30:0: C0116: Missing function or method docstring (missing-function-docstring)
actualizadores.py:38:0: C0116: Missing function or method docstring (missing-function-docstring)
actualizadores.py:50:0: C0116: Missing function or method docstring (missing-function-docstring)
actualizadores.py:61:0: C0116: Missing function or method docstring (missing-function-docstring)
************* Module texttest_fixture
texttest_fixture.py:1:0: C0114: Missing module docstring (missing-module-docstring)
texttest_fixture.py:4:0: W0401: Wildcard import gilded_rose (wildcard-import)
texttest_fixture.py:7:0: C0116: Missing function or method docstring (missing-function-docstring)
texttest_fixture.py:21:4: C0415: Import outside toplevel (sys) (import-outside-toplevel)
texttest_fixture.py:25:14: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
texttest_fixture.py:4:0: W0614: Unused import(s) actualizar_item from wildcard import of gilded_rose (unused-wildcard-
import)

-------------------------------------------------------------------
Your code has been rated at 6.94/10 (previous run: 6.48/10, +0.46)

C:\Users\manul\AppData\Local\Programs\Python\Python312\Lib\site-packages\pylint\__init__.py:34: ReporterWarning: Both
NO_COLOR and FORCE_COLOR are set! (disabling colors)
  PylintRun(argv or sys.argv[1:])

EXIT=28
```

Figura 6: Salida de pylint después de la refactorización.

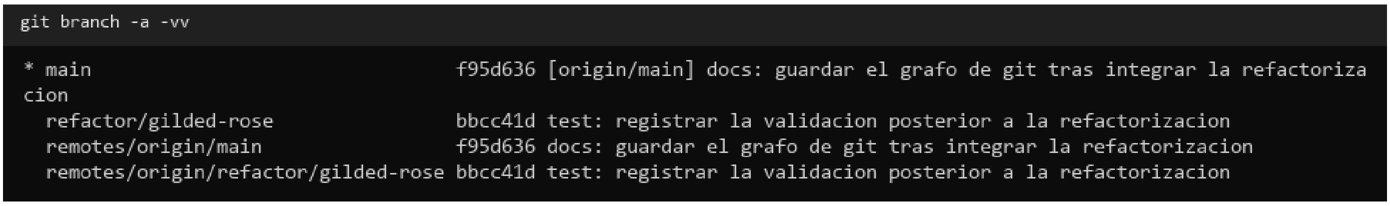


```
lizard -C 10 -L 50 gilded_rose.py actualizadores.py texttest_fixture.py
```

```
================================================
  NLOC    CCN   token  PARAM  length  location
------------------------------------------------
       2      1     12      2       2 __init__@8-9@gilded_rose.py
       3      2     16      1       3 update_quality@11-13@gilded_rose.py
       4      1     26      4       4 __init__@17-20@gilded_rose.py
       2      1     21      1       2 __repr__@22-23@gilded_rose.py
       3      2     21      1       3 bajar_calidad@12-14@actualizadores.py
       3      2     21      1       3 subir_calidad@17-19@actualizadores.py
       6      2     30      1       6 actualizar_normal@22-27@actualizadores.py
       6      2     30      1       6 actualizar_brie@30-35@actualizadores.py
      10      4     59      1      10 actualizar_backstage@38-47@actualizadores.py
       2      1      6      1       2 actualizar_sulfuras@50-51@actualizadores.py
       2      1     18      1       2 actualizar_item@61-62@actualizadores.py
      24      4    215      0      24 main@7-30@texttest_fixture.py
3 file analyzed.
==============================================================
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt    file
--------------------------------------------------------------
     14       2.8     1.2       18.8         4     gilded_rose.py
     44       4.6     2.0       26.4         7     actualizadores.py
     28      24.0     4.0      215.0         1     texttest_fixture.py

===============================================================================================
No thresholds exceeded (cyclomatic_complexity > 10 or length > 50 or nloc > 1000000 or parameter_count > 100)
==========================================================================================
Total nloc   Avg.NLOC  AvgCCN  Avg.token   Fun Cnt  Warning cnt   Fun Rt   nloc Rt
------------------------------------------------------------------------------------------
        86       5.6     1.9       39.6       12            0      0.00    0.00

EXIT=0
```

Figura 7: Salida de lizard después de la refactorización.

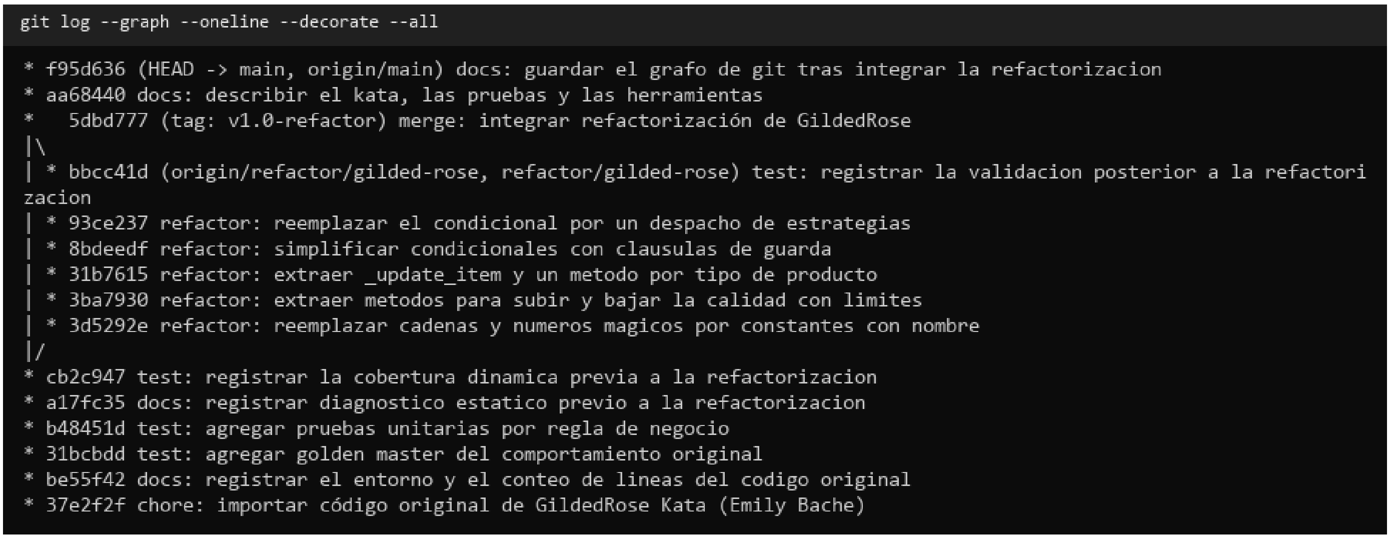
`refactorización de GildedRose`, con dos padres. El tag `v1.0-refactor` apunta a ese merge. Figuras 8 y 9. La página del repositorio y el listado de ramas están en las figuras 10 y 11.



```
git branch -a -vv
```

```
* main                              f95d636 [origin/main] docs: guardar el grafo de git tras integrar la refactoriza
cion
  refactor/gilded-rose              bbcc41d test: registrar la validacion posterior a la refactorizacion
  remotes/origin/main               f95d636 docs: guardar el grafo de git tras integrar la refactorizacion
  remotes/origin/refactor/gilded-rose bbcc41d test: registrar la validacion posterior a la refactorizacion
```

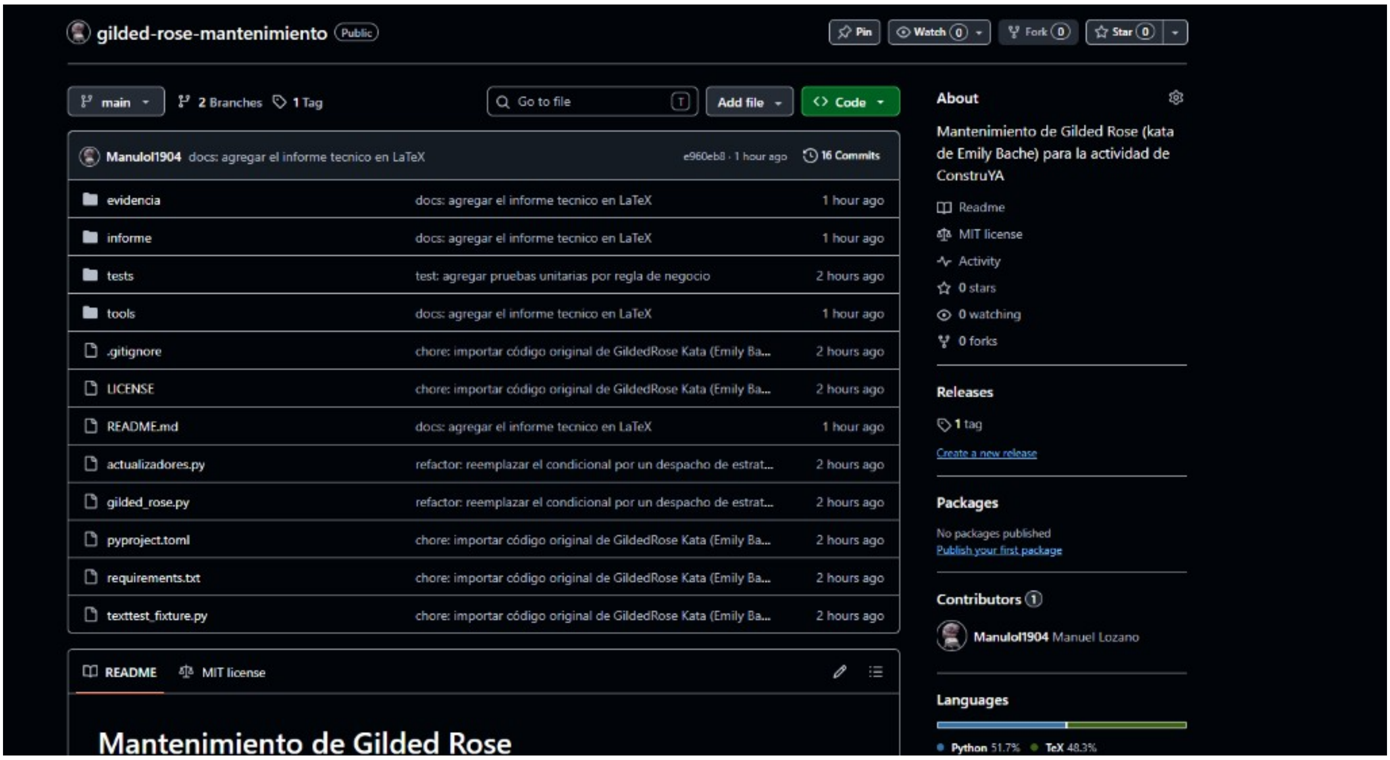
Figura 8: Salida de `git branch -a -vv`.



```
git log --graph --oneline --decorate --all
```

```
* f95d636 (HEAD -> main, origin/main) docs: guardar el grafo de git tras integrar la refactorizacion
* aa68440 docs: describir el kata, las pruebas y las herramientas
*   5dbd777 (tag: v1.0-refactor) merge: integrar refactorización de GildedRose
|\
| * bbcc41d (origin/refactor/gilded-rose, refactor/gilded-rose) test: registrar la validacion posterior a la refactori
zacion
| * 93ce237 refactor: reemplazar el condicional por un despacho de estrategias
| * 8bdeedf refactor: simplificar condicionales con clausulas de guarda
| * 31b7615 refactor: extraer _update_item y un metodo por tipo de producto
| * 3ba7930 refactor: extraer metodos para subir y bajar la calidad con limites
| * 3d5292e refactor: reemplazar cadenas y numeros magicos por constantes con nombre
|/
* cb2c947 test: registrar la cobertura dinamica previa a la refactorizacion
* a17fc35 docs: registrar diagnostico estatico previo a la refactorizacion
* b48451d test: agregar pruebas unitarias por regla de negocio
* 31bcbdd test: agregar golden master del comportamiento original
* be55f42 docs: registrar el entorno y el conteo de lineas del codigo original
* 37e2f2f chore: importar código original de GildedRose Kata (Emily Bache)
```

Figura 9: Salida de `git log -graph -oneline -decorate -all`.



Manulol1904 docs: agregar el informe tecnico en LaTeX e960eb8 · 1 hour ago 16 Commits

evidencia	docs: agregar el informe tecnico en LaTeX	1 hour ago
informe	docs: agregar el informe tecnico en LaTeX	1 hour ago
tests	test: agregar pruebas unitarias por regla de negocio	2 hours ago
tools	docs: agregar el informe tecnico en LaTeX	1 hour ago
.gitignore	chore: importar código original de GildedRose Kata (Emily Ba...	2 hours ago
LICENSE	chore: importar código original de GildedRose Kata (Emily Ba...	2 hours ago
README.md	docs: agregar el informe tecnico en LaTeX	1 hour ago
actualizadores.py	refactor: reemplazar el condicional por un despacho de estrat...	2 hours ago
gilded_rose.py	refactor: reemplazar el condicional por un despacho de estrat...	2 hours ago
pyproject.toml	chore: importar código original de GildedRose Kata (Emily Ba...	2 hours ago
requirements.txt	chore: importar código original de GildedRose Kata (Emily Ba...	2 hours ago
texttest_fixture.py	chore: importar código original de GildedRose Kata (Emily Ba...	2 hours ago

Mantenimiento de Gilded Rose

About

Mantenimiento de Gilded Rose (kata de Emily Bache) para la actividad de ConstruYA

Readme
MIT license
Activity
0 stars
0 watching
0 forks

Releases

1 tag
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

Manulol1904 Manuel Lozano

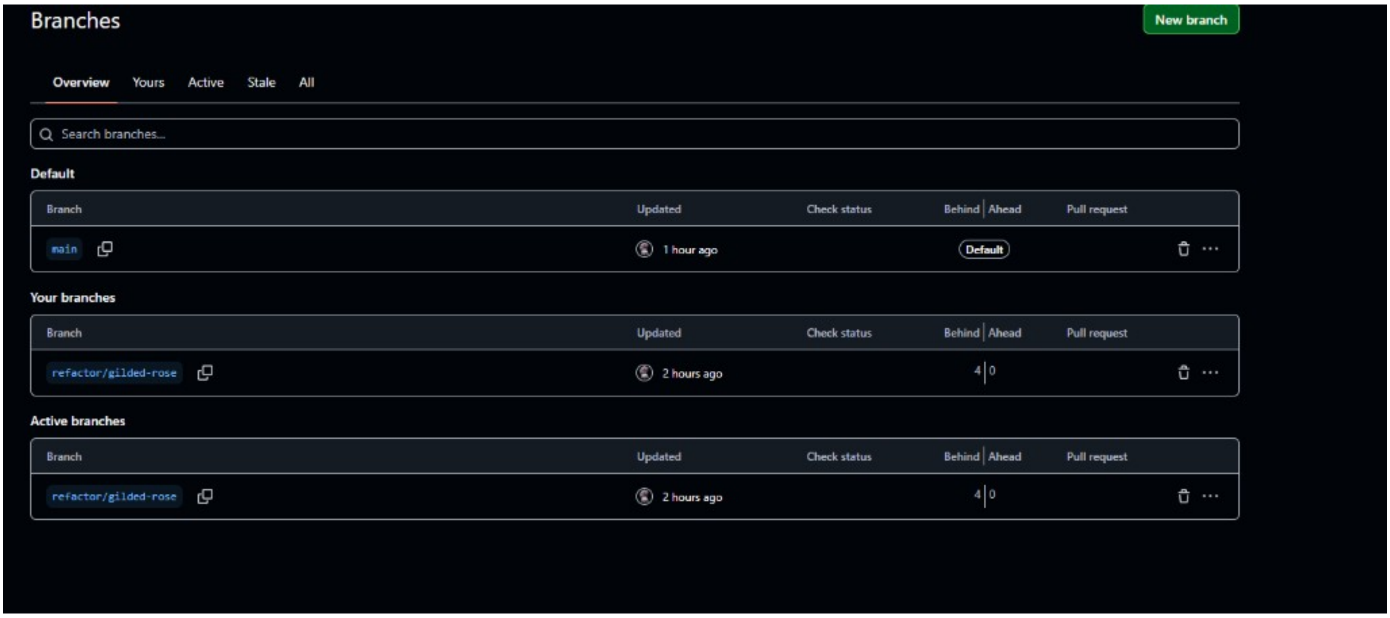
Languages

Python 51.7% TeX 48.3%

Figura 10: Captura manual de la página del repositorio público en GitHub.

6. Pruebas y validación

unittest, 11 pruebas de regla y 2 de golden master. El ítem normal baja 1; vencido, baja 2; la calidad no pasa de 0 ni de 50; Aged Brie sube; Sulfuras no cambia; Backstage sube 1, 2 o 3 según le falten más de 10, 10 o menos, o 5 o menos días, y cae a 0 tras el concierto. "Conjured Mana Cake" baja 1, igual que un ítem normal. El golden master compara 5833 combinaciones y 40 días contra `tests/golden_master.json`. `git diff` de ese archivo no tiene líneas (figura 14). Antes y después, las 13 pruebas pasan (figuras 12 y 13).



Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	1 hour ago		Default		

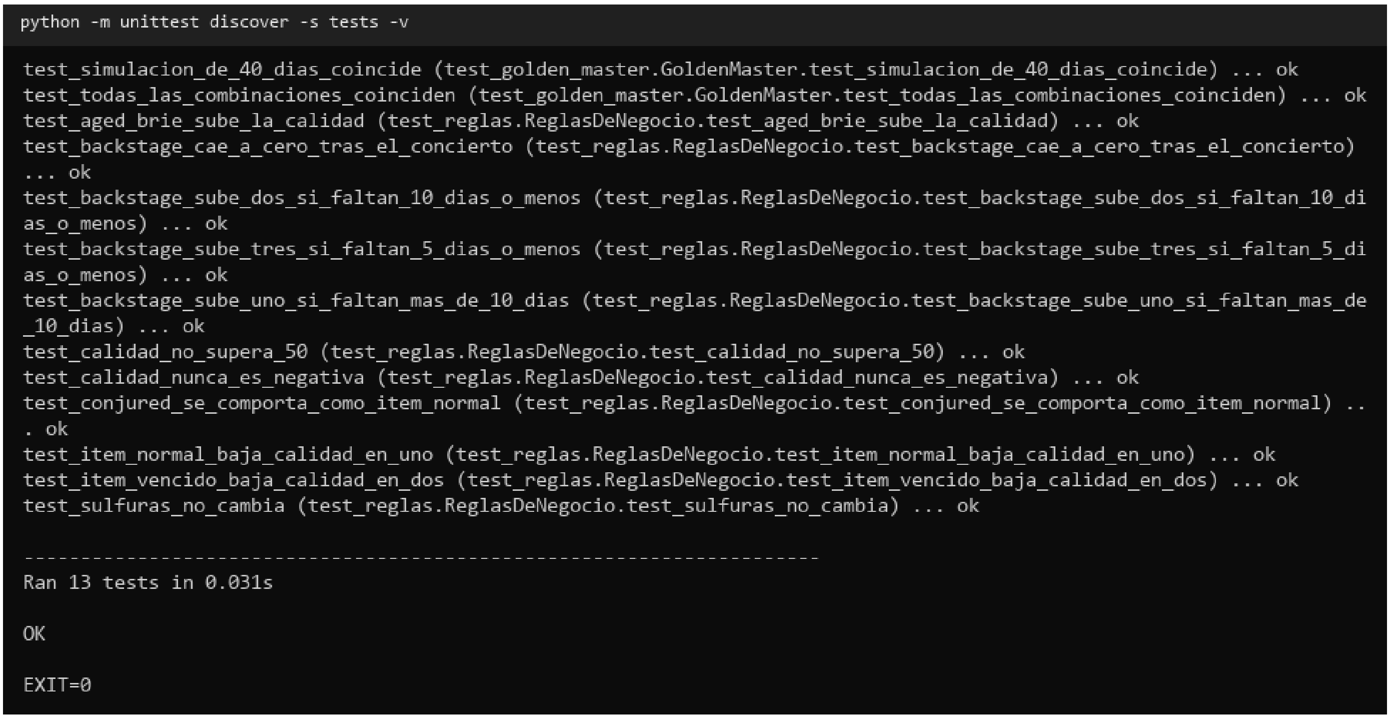
Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
refactor/gilded-rose	2 hours ago		4	0	

Active branches

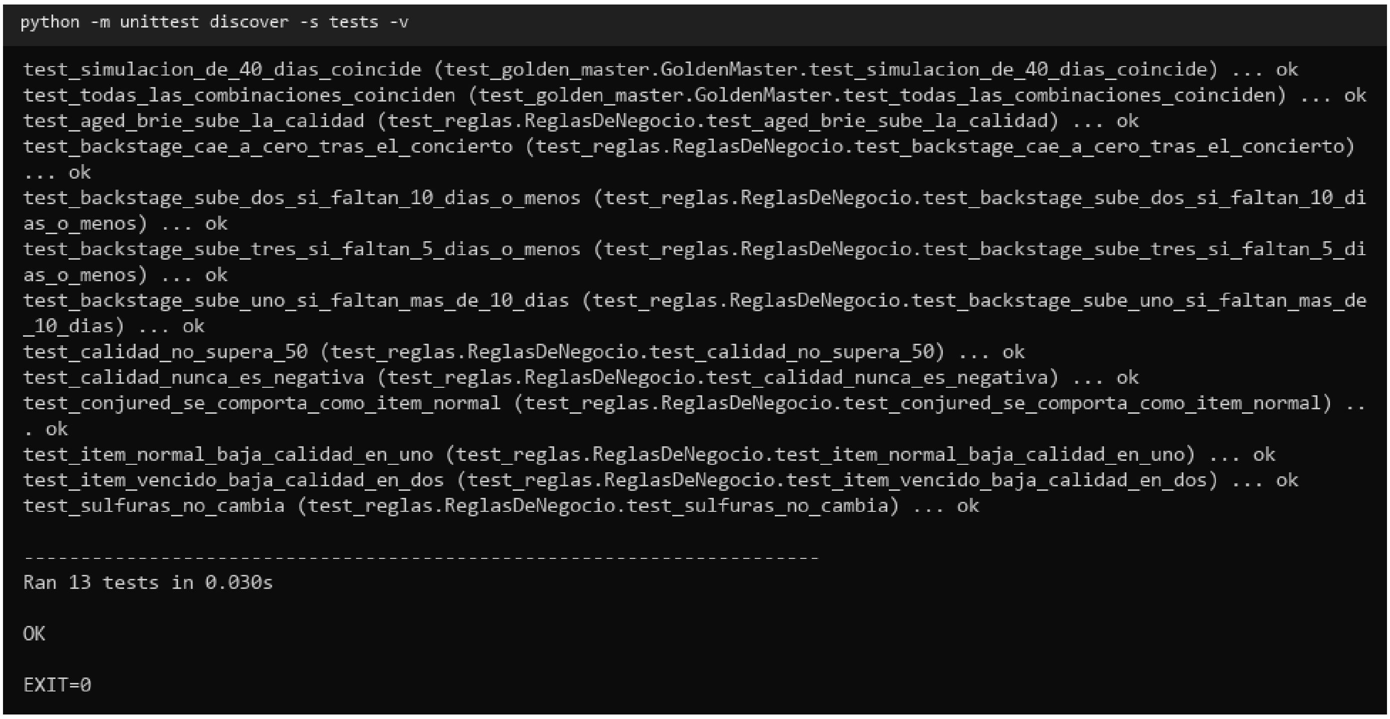
Branch	Updated	Check status	Behind	Ahead	Pull request
refactor/gilded-rose	2 hours ago		4	0	

Figura 11: Captura manual de las ramas `main` y `refactor/gilded-rose` en GitHub.



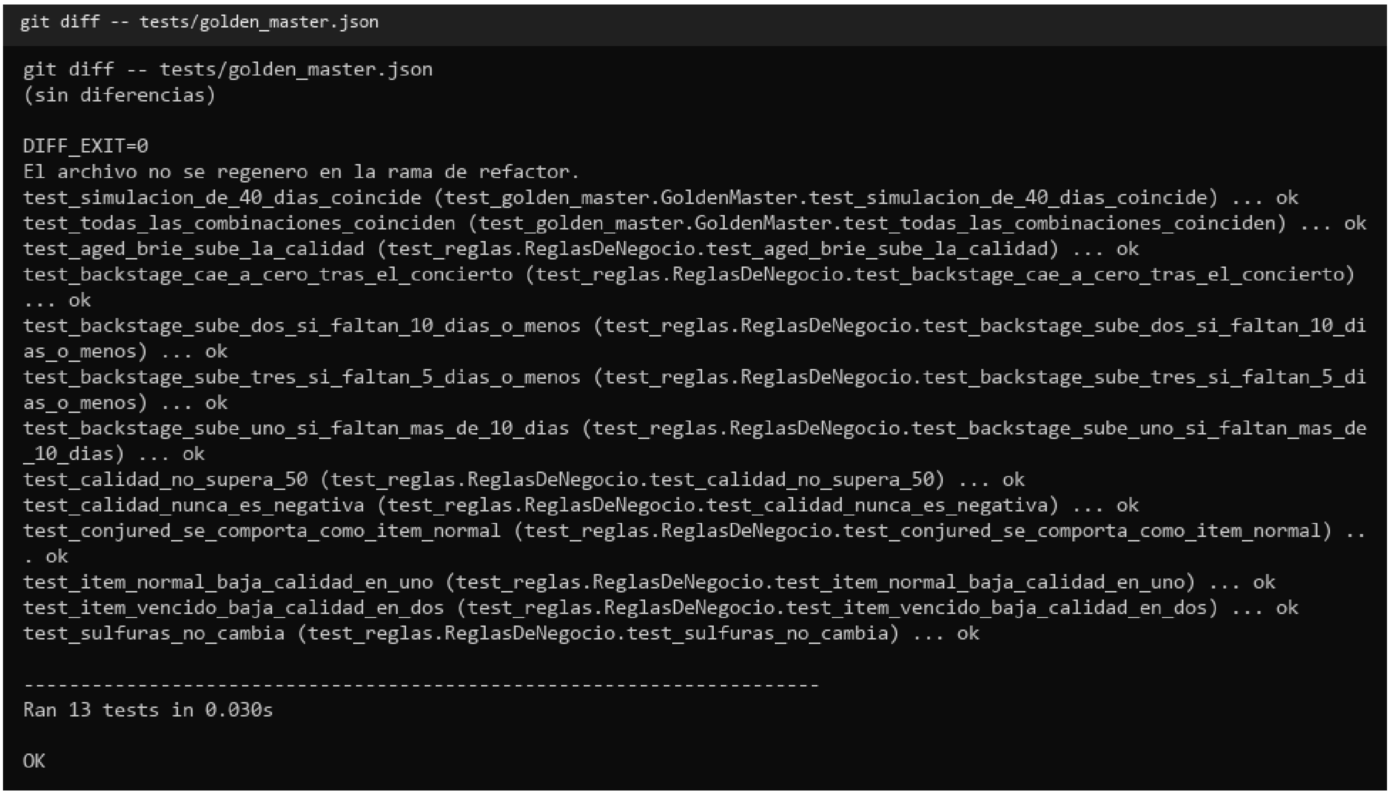
```
python -m unittest discover -s tests -v

test_simulacion_de_40_dias_coincide (test_golden_master.GoldenMaster.test_simulacion_de_40_dias_coincide) ... ok
test_todas_las_combinaciones_coinciden (test_golden_master.GoldenMaster.test_todas_las_combinaciones_coinciden) ... ok
test_aged_brie_sube_la_calidad (test_reglas.ReglasDeNegocio.test_aged_brie_sube_la_calidad) ... ok
test_backstage_cae_a_cero_tras_el_concierto (test_reglas.ReglasDeNegocio.test_backstage_cae_a_cero_tras_el_concierto)
... ok
test_backstage_sube_dos_si_faltan_10_dias_o_menos (test_reglas.ReglasDeNegocio.test_backstage_sube_dos_si_faltan_10_di
as_o_menos) ... ok
test_backstage_sube_tres_si_faltan_5_dias_o_menos (test_reglas.ReglasDeNegocio.test_backstage_sube_tres_si_faltan_5_di
as_o_menos) ... ok
test_backstage_sube_uno_si_faltan_mas_de_10_dias (test_reglas.ReglasDeNegocio.test_backstage_sube_uno_si_faltan_mas_de
_10_dias) ... ok
test_calidad_no_supera_50 (test_reglas.ReglasDeNegocio.test_calidad_no_supera_50) ... ok
test_calidad_nunca_es_negativa (test_reglas.ReglasDeNegocio.test_calidad_nunca_es_negativa) ... ok
test_conjured_se_comporta_como_item_normal (test_reglas.ReglasDeNegocio.test_conjured_se_comporta_como_item_normal) ..
. ok
test_item_normal_baja_calidad_en_uno (test_reglas.ReglasDeNegocio.test_item_normal_baja_calidad_en_uno) ... ok
test_item_vencido_baja_calidad_en_dos (test_reglas.ReglasDeNegocio.test_item_vencido_baja_calidad_en_dos) ... ok
test_sulfuras_no_cambia (test_reglas.ReglasDeNegocio.test_sulfuras_no_cambia) ... ok

----------------------------------------------------------------------
Ran 13 tests in 0.031s

OK

EXIT=0
```

Figura 12: Salida de `python -m unittest discover -s tests -v` antes de refactorizar.



```
python -m unittest discover -s tests -v

test_simulacion_de_40_dias_coincide (test_golden_master.GoldenMaster.test_simulacion_de_40_dias_coincide) ... ok
test_todas_las_combinaciones_coinciden (test_golden_master.GoldenMaster.test_todas_las_combinaciones_coinciden) ... ok
test_aged_brie_sube_la_calidad (test_reglas.ReglasDeNegocio.test_aged_brie_sube_la_calidad) ... ok
test_backstage_cae_a_cero_tras_el_concierto (test_reglas.ReglasDeNegocio.test_backstage_cae_a_cero_tras_el_concierto)
... ok
test_backstage_sube_dos_si_faltan_10_dias_o_menos (test_reglas.ReglasDeNegocio.test_backstage_sube_dos_si_faltan_10_di
as_o_menos) ... ok
test_backstage_sube_tres_si_faltan_5_dias_o_menos (test_reglas.ReglasDeNegocio.test_backstage_sube_tres_si_faltan_5_di
as_o_menos) ... ok
test_backstage_sube_uno_si_faltan_mas_de_10_dias (test_reglas.ReglasDeNegocio.test_backstage_sube_uno_si_faltan_mas_de
_10_dias) ... ok
test_calidad_no_supera_50 (test_reglas.ReglasDeNegocio.test_calidad_no_supera_50) ... ok
test_calidad_nunca_es_negativa (test_reglas.ReglasDeNegocio.test_calidad_nunca_es_negativa) ... ok
test_conjured_se_comporta_como_item_normal (test_reglas.ReglasDeNegocio.test_conjured_se_comporta_como_item_normal) ..
. ok
test_item_normal_baja_calidad_en_uno (test_reglas.ReglasDeNegocio.test_item_normal_baja_calidad_en_uno) ... ok
test_item_vencido_baja_calidad_en_dos (test_reglas.ReglasDeNegocio.test_item_vencido_baja_calidad_en_dos) ... ok
test_sulfuras_no_cambia (test_reglas.ReglasDeNegocio.test_sulfuras_no_cambia) ... ok

----------------------------------------------------------------------
Ran 13 tests in 0.030s

OK

EXIT=0
```

Figura 13: Salida del mismo comando de unittest después de refactorizar.



```
git diff -- tests/golden_master.json

git diff -- tests/golden_master.json
(sin diferencias)

DIFF_EXIT=0
El archivo no se regenero en la rama de refactor.
test_simulacion_de_40_dias_coincide (test_golden_master.GoldenMaster.test_simulacion_de_40_dias_coincide) ... ok
test_todas_las_combinaciones_coinciden (test_golden_master.GoldenMaster.test_todas_las_combinaciones_coinciden) ... ok
test_aged_brie_sube_la_calidad (test_reglas.ReglasDeNegocio.test_aged_brie_sube_la_calidad) ... ok
test_backstage_cae_a_cero_tras_el_concierto (test_reglas.ReglasDeNegocio.test_backstage_cae_a_cero_tras_el_concierto)
... ok
test_backstage_sube_dos_si_faltan_10_dias_o_menos (test_reglas.ReglasDeNegocio.test_backstage_sube_dos_si_faltan_10_di
as_o_menos) ... ok
test_backstage_sube_tres_si_faltan_5_dias_o_menos (test_reglas.ReglasDeNegocio.test_backstage_sube_tres_si_faltan_5_di
as_o_menos) ... ok
test_backstage_sube_uno_si_faltan_mas_de_10_dias (test_reglas.ReglasDeNegocio.test_backstage_sube_uno_si_faltan_mas_de
_10_dias) ... ok
test_calidad_no_supera_50 (test_reglas.ReglasDeNegocio.test_calidad_no_supera_50) ... ok
test_calidad_nunca_es_negativa (test_reglas.ReglasDeNegocio.test_calidad_nunca_es_negativa) ... ok
test_conjured_se_comporta_como_item_normal (test_reglas.ReglasDeNegocio.test_conjured_se_comporta_como_item_normal) ..
. ok
test_item_normal_baja_calidad_en_uno (test_reglas.ReglasDeNegocio.test_item_normal_baja_calidad_en_uno) ... ok
test_item_vencido_baja_calidad_en_dos (test_reglas.ReglasDeNegocio.test_item_vencido_baja_calidad_en_dos) ... ok
test_sulfuras_no_cambia (test_reglas.ReglasDeNegocio.test_sulfuras_no_cambia) ... ok

----------------------------------------------------------------------
Ran 13 tests in 0.030s

OK
```

Figura 14: Salida de `git diff - tests/golden_master.json`: el archivo no cambió.

7. Limitaciones y trabajo futuro

No se implementó la regla Conjured: hoy ese nombre sigue el camino del ítem normal, y cambiarlo modificaría el comportamiento fijado por las pruebas. `Item` sigue siendo la clase original. `texttest_fixture.py` conserva el import estrella. Las funciones nuevas no tienen docstring, y por eso pylint cuenta más avisos aunque la nota sube.

8. Reflexión sobre calidad y sostenibilidad

El cambio baja el costo de una regla nueva. Aged Brie, Sulfuras y Backstage ya no comparten un condicional de seis niveles: cada uno tiene una función, y el golden master avisa si la salida se mueve. Eso reduce el riesgo de una falla al mantener el inventario. El límite es el que las pruebas fijan el comportamiento actual, incluido el de Conjured. Una solicitud equivalente a la pasarela de pagos de ConstruYA, por ejemplo otra familia de productos, debería entrar como otra estrategia y con pruebas propias, no como otra rama dentro de `update_quality`. La deuda que queda está localizada: la clase `Item` no se toca y la regla Conjured sigue pendiente.

9. Contraste con la rúbrica

Criterio	Dónde se evidencia
C1 Diagnóstico	Sección 2, figuras 2 y 3, `evidencia/diagnostico_smells.txt`. Seis smells.
C2 Refactorización	Sección 3. Cinco técnicas, antes y después. El golden master no cambió.
C3 Herramientas	Sección 4. Estática: pylint y lizard. Dinámica: coverage.py con ramas.
C4 Git	Sección 5, figuras 8, 9, 10 y 11. Rama, commits atómicos, merge `5dbd777` y tag `v1.0-refactor`.
C5 Pruebas	Sección 6, figuras 12, 13 y 14. Trece pruebas y diff vacío.